

Prolog

Un langage mature mais sous-estimé pour de l'IA symbolique !

Frédéric Cabestre & Didier Plaindoux

DEVOXX France 2026

Pourquoi un tel titre ?

Approche Connexionniste

Le connexionnisme met en avant l'idée que c'est en entraînant la machine à apprendre qu'elle sera en mesure d'agir de manière intelligente.

Approche Connexionniste

Le connexionnisme met en avant l'idée que c'est en entraînant la machine à apprendre qu'elle sera en mesure d'agir de manière intelligente.

L'approche connexionniste a souvent été critiquée pour son opacité.

Pourquoi un tel titre ?

Approche Connexionniste

Le connexionnisme met en avant l'idée que c'est en entraînant la machine à apprendre qu'elle sera en mesure d'agir de manière intelligente.

L'approche connexionniste a souvent été critiquée pour son opacité.

Approche Symbolique

Technique qui s'appuie sur la logique et la manipulation de symboles. Son application la plus connue est la conception des systèmes experts

Pourquoi un tel titre ?

Approche Connexionniste

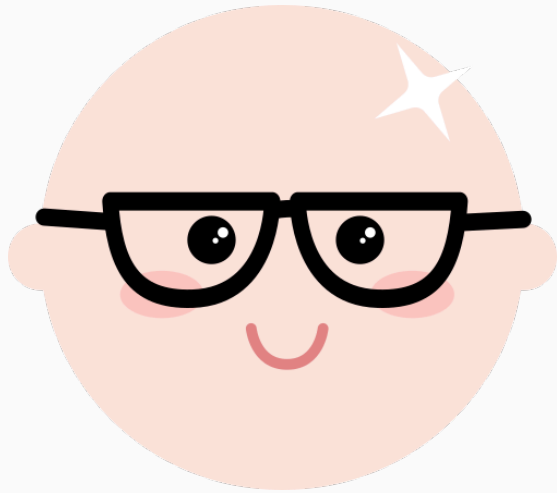
Le connexionnisme met en avant l'idée que c'est en entraînant la machine à apprendre qu'elle sera en mesure d'agir de manière intelligente.

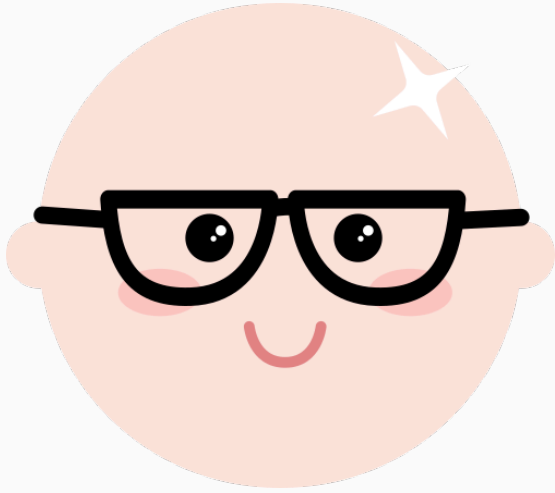
L'approche connexionniste a souvent été critiquée pour son opacité.

Approche Symbolique

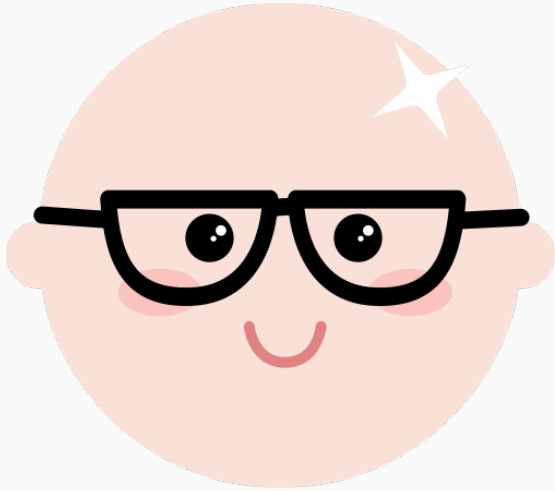
Technique qui s'appuie sur la logique et la manipulation de symboles. Son application la plus connue est la conception des systèmes experts

L'intelligence artificielle symbolique est une intelligence «lisible» par l'homme.





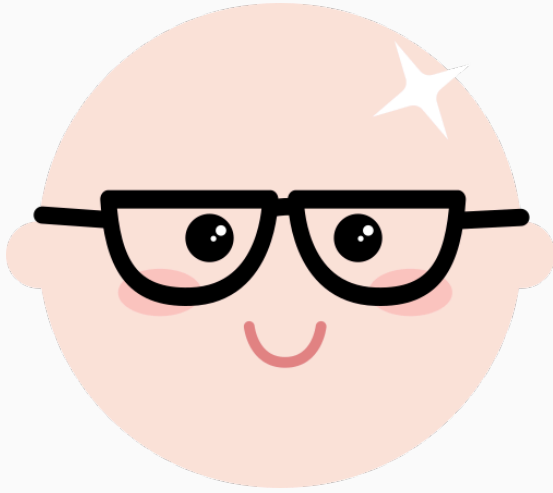
Artisan du logiciel revendiqué, issu du monde académique.



Artisan du logiciel revendiqué, issu du monde académique.

Mobile Device Management

Bravas est un système de gestion unifiée des appareils, des identités et de la sécurité, conçue pour les PME.

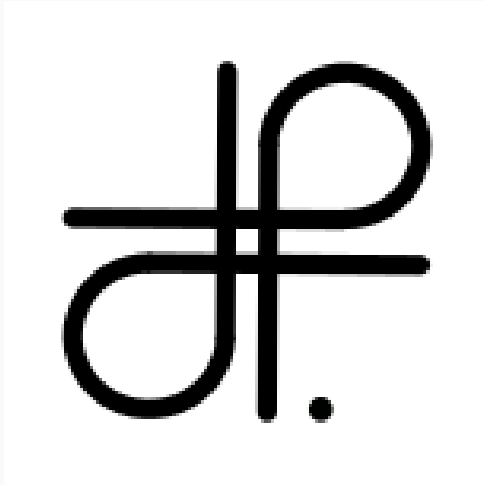


Artisan du logiciel revendiqué, issu du monde académique.

Mobile Device Management

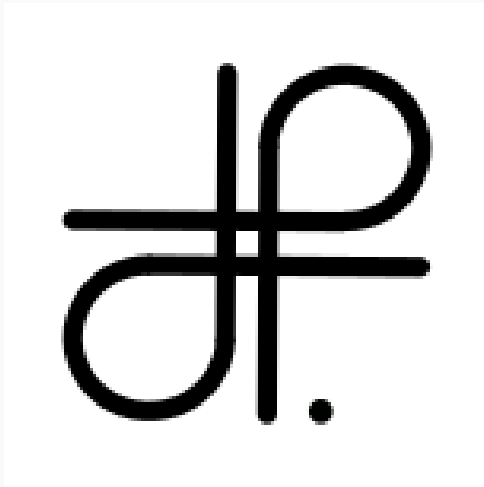
Bravas est un système de gestion unifiée des appareils, des identités et de la sécurité, conçue pour les PME.

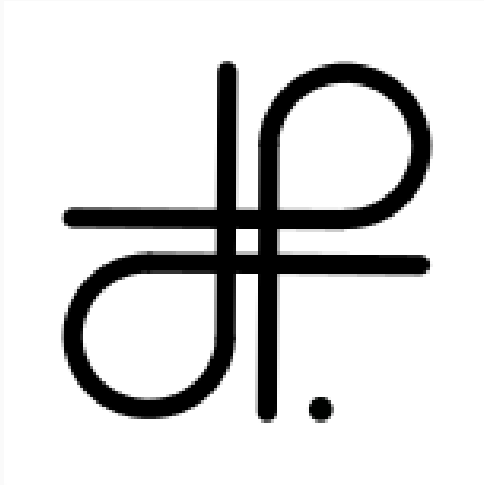
Bravas vient d'être acquis par Remote, solution de RH et de paie internationale.



Système Distribué et modèle Acteur 🚀

Akawan Kaptngo est un système distribué sécurisé pour des exécutions décentralisées avec des communications optimisées.



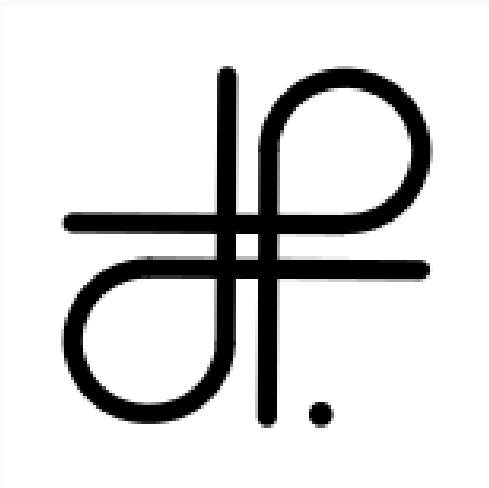


Système Distribué et modèle Acteur 🚀

Akawan Kaptngo est un système distribué sécurisé pour des exécutions décentralisées avec des communications optimisées.

Système Expert en Prolog 🎬

Crédifix est un système de mise en relation et de négociation immobilière intégrant des critères de durabilité.



Système Distribué et modèle Acteur 🚀

Akawan Kaptngo est un système distribué sécurisé pour des exécutions décentralisées avec des communications optimisées.

Système Expert en Prolog 🎬

Crédifix est un système de mise en relation et de négociation immobilière intégrant des critères de durabilité.

Mobilité de Code 🚧 [OSS]

Ephel est un langage qui combine :

- la programmation fonctionnelle et
- le calcul des ambients

Dans la suite de la présentation on nomme terme:

Dans la suite de la présentation on nomme terme:

- un littéral (entier, caractère etc.) e.g. 1, 'a'

Dans la suite de la présentation on nomme terme:

- ▶ un littéral (entier, caractère etc.) e.g. 1, 'a'
- ▶ un atome (commence par une minuscule) e.g. albert, γ

Dans la suite de la présentation on nomme terme:

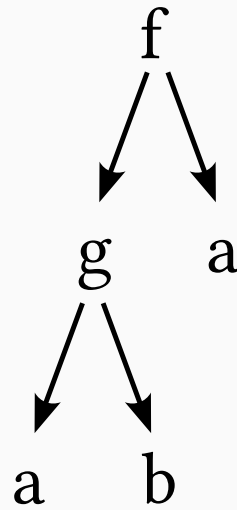
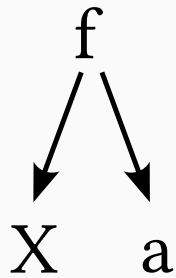
- ▶ un littéral (entier, caractère etc.) e.g. 1, 'a'
- ▶ un atome (commence par une minuscule) e.g. albert, γ
- ▶ un foncteur e.g. **personne**(robert, smith), f("Hello",1)

Dans la suite de la présentation on nomme terme:

- ▶ un littéral (entier, caractère etc.) e.g. 1, 'a'
- ▶ un atome (commence par une minuscule) e.g. albert, γ
- ▶ un foncteur e.g. personne(robert, smith), f("Hello",1)
- ▶ une variable (commence par une majuscule) e.g. X, Γ

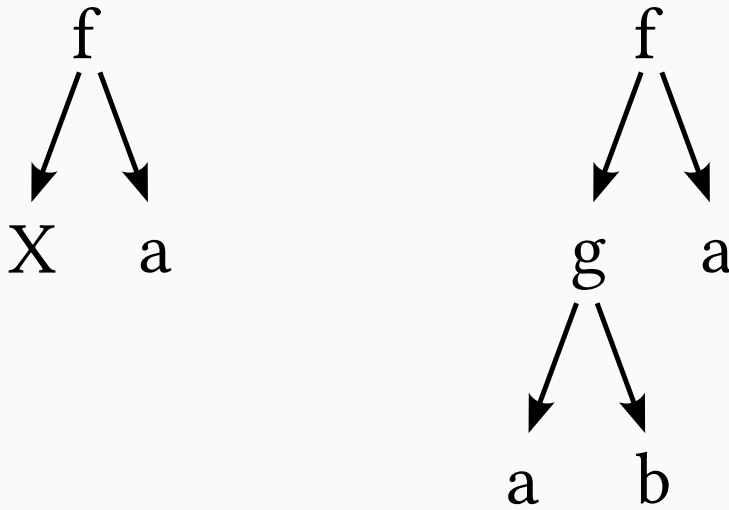
Convention : Visualisation des Termes

Les termes peuvent être vus comme des graphes orientés.



Convention : Visualisation des Termes

Les termes peuvent être vus comme des graphes orientés.



Dénotent les termes **$f(X, a)$** et **$f(g(a, b), a)$** .

Principe: Le filtrage de motifs



Filtrage de Motifs (ou Pattern Matching)

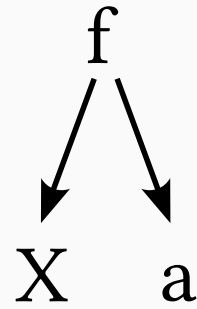
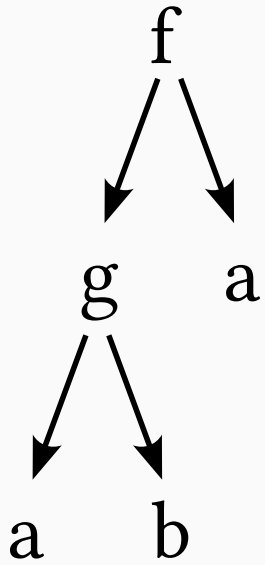
Filtrage de Motifs (ou Pattern Matching)

- Présent dans OCaml, Haskell, Rust ou Java par exemple.

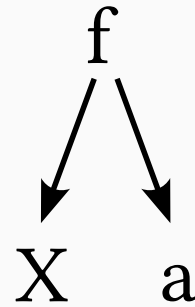
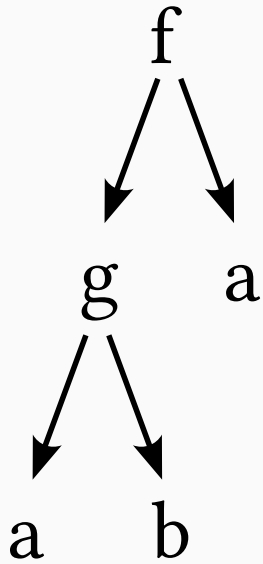
Filtrage de Motifs (ou Pattern Matching)

- ▶ Présent dans OCaml, Haskell, Rust ou Java par exemple.
- ▶ Capturer des valeurs d'un terme en utilisant un **motif**.

Filtrage : Principe

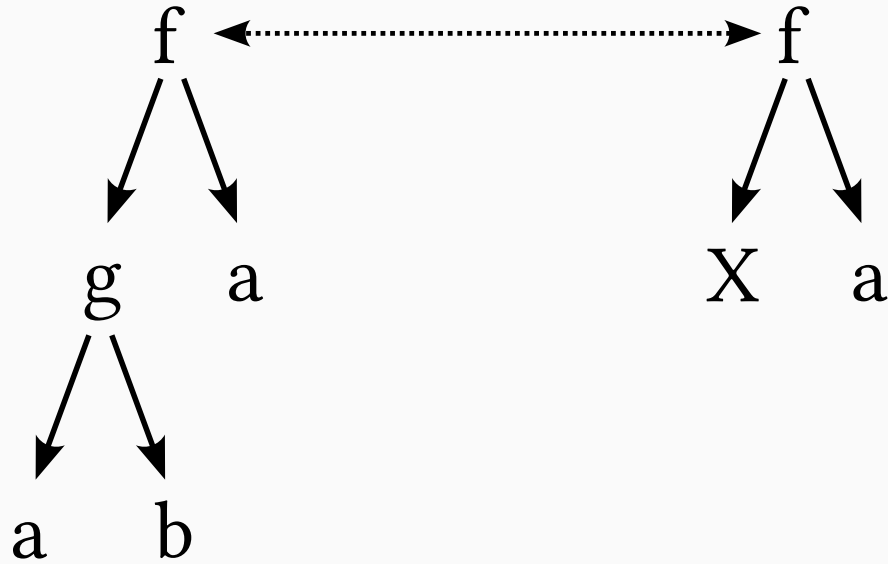


Filtrage : Principe



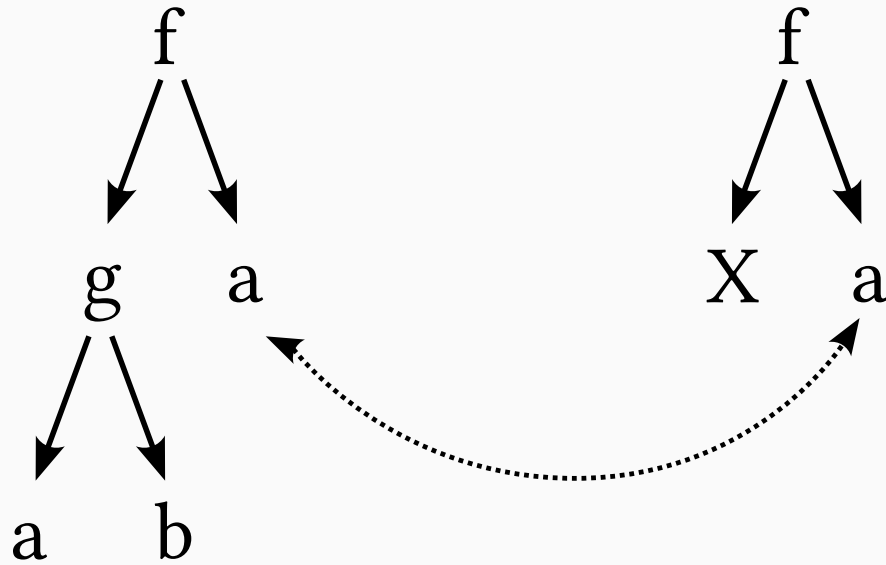
Ici on a un terme $f(g(a, b), a)$ et on veut le filtrer avec $f(X, a)$ ou X est une variable.

Filtrage : Principe



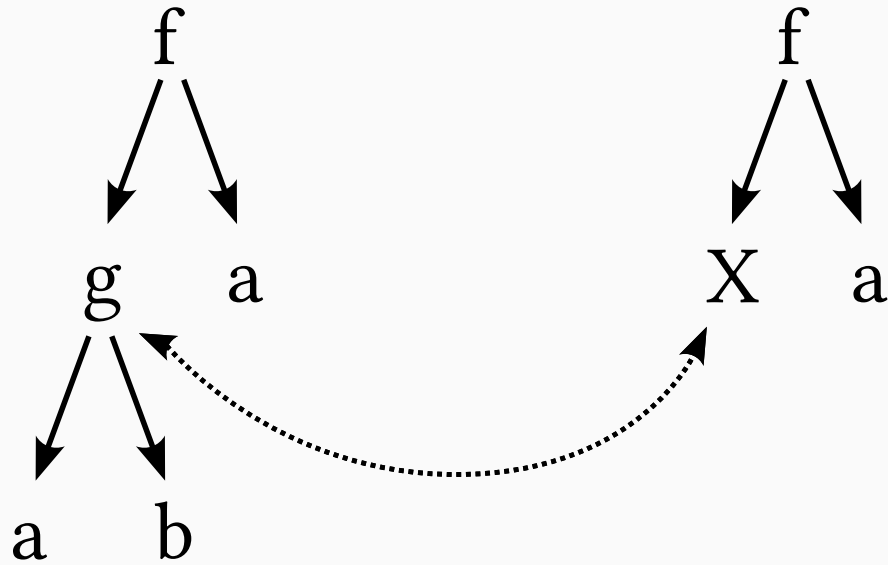
Les foncteurs de plus haut niveau correspondent.

Filtrage : Principe

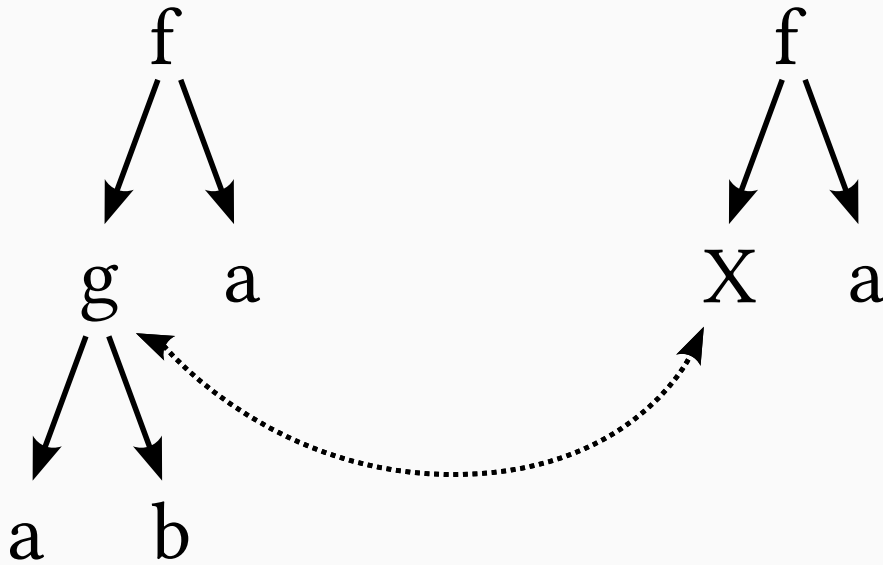


Les constantes en seconde position correspondent.

Filtrage : Principe



Filtrage : Principe



Associe la variable **X** au terme **g(a,b)** afin de rendre le filtre identique au terme.

Filtrage : Et en Java ?

```
sealed interface Nat {  
    record Zero() implements Nat {}  
    record Succ(Nat value) implements Nat {}  
  
    default Nat add(Nat n) {  
        return switch (this) {  
            case Zero() -> n;  
            case Succ(var p) -> new Succ(p.add(n));  
        };  
    }  
}
```

Filtrage : Et en Java ?

```
sealed interface Nat {  
    record Zero() implements Nat {}  
    record Succ(Nat value) implements Nat {}  
  
    default Nat add(Nat n) {  
        return switch (this) {  
            case Zero() -> n;  
            case Succ(var p) -> new Succ(p.add(n));  
        };  
    }  
}
```

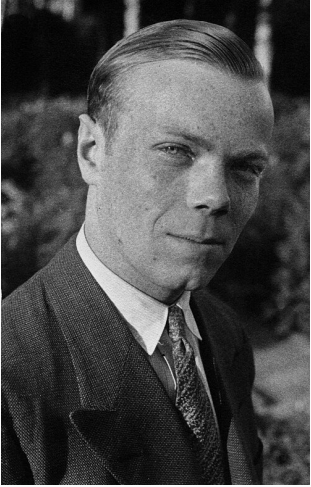
Le motif **Succ(var p)** permet de capturer le prédécesseur en le liant à la variable **p**

Filtrage : Limitations

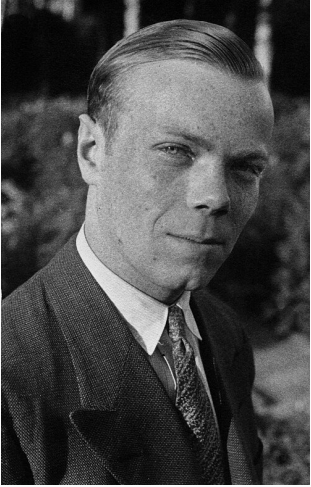
- Le motif peut contenir des variables non répétées, p.ex. $f(x, x)$ est interdit.

- ▶ Le motif peut contenir des variables non répétées, p.ex. $f(X, X)$ est interdit.
- ▶ Le terme filtré ne doit pas contenir de variables: il est dit fermé.

Fondations: L'unification

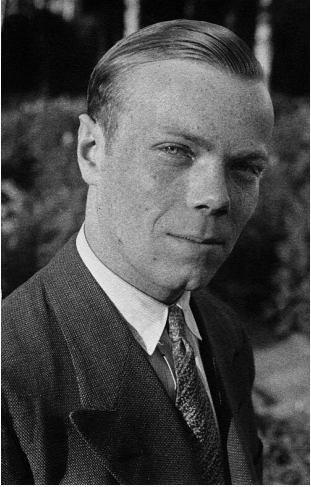


Jacques Herbrand (1908-1931), mathématicien et logicien français



Jacques Herbrand (1908-1931), mathématicien et logicien français

Il propose une méthode pour déterminer si on peut trouver une substitution telle qu'elle rende deux termes identiques.



Jacques Herbrand (1908-1931), mathématicien et logicien français

Il propose une méthode pour déterminer si on peut trouver une substitution telle qu'elle rende deux termes identiques.

Une telle substitution est notée σ

Unification vs. Filtrage

Unification vs. Filtrage

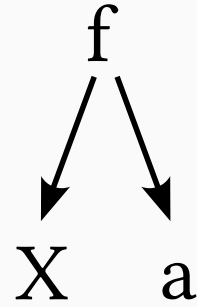
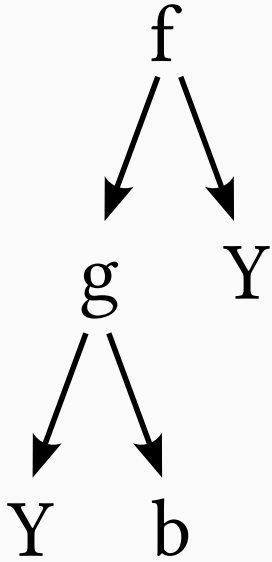
- ▶ *le motif peut contenir des variables non répétées, p.ex. $f(X,X)$ est interdit.*
- ▶ *Le terme filtré ne doit pas contenir de variables: il est dit fermé.*

- ▶ Tout terme peut contenir des variables répétées, p.ex. $f(x, x)$.
- ▶ *Le terme filtré ne doit pas contenir de variables: il est dit fermé.*

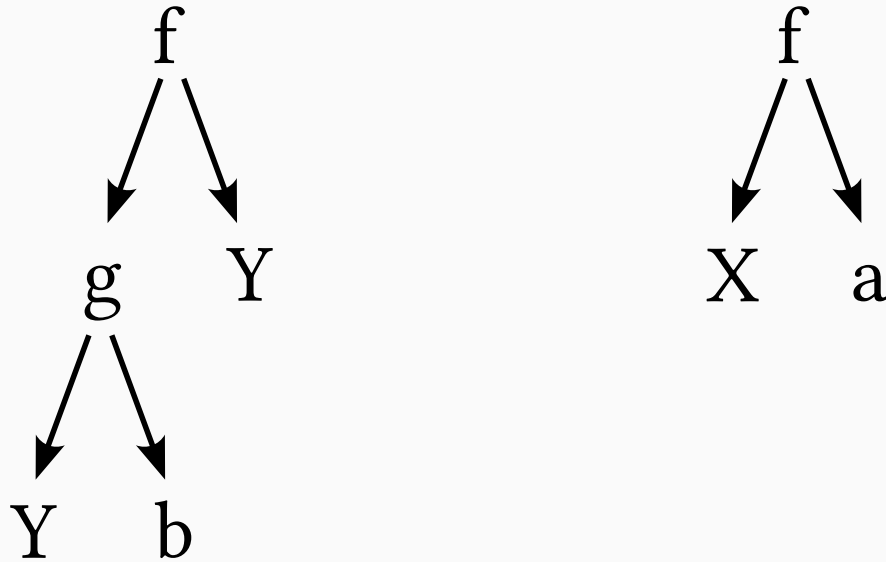
Unification vs. Filtrage

- ▶ Tout terme peut contenir des variables répétées, p.ex. $f(x, x)$.
- ▶ L'unification est faite entre deux termes. Les variables sont des termes !

Unification : Principe

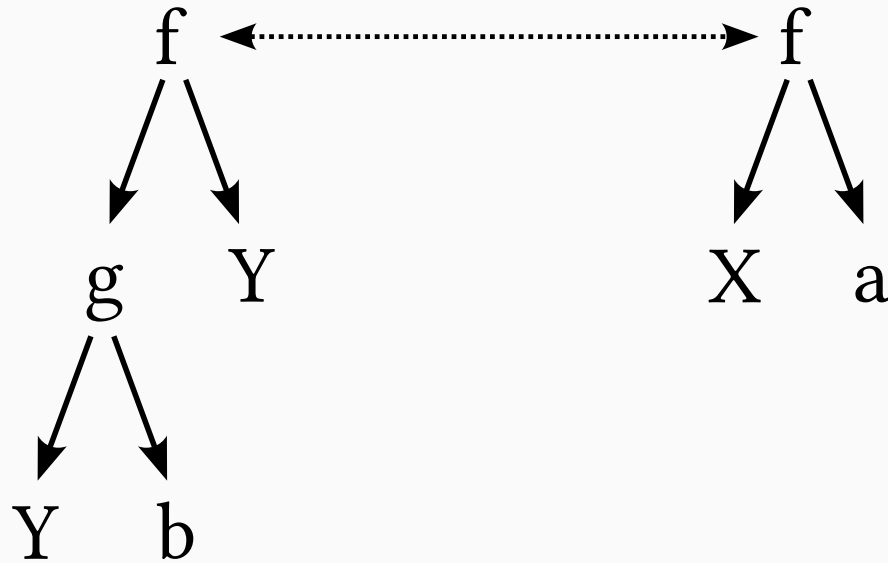


Unification : Principe



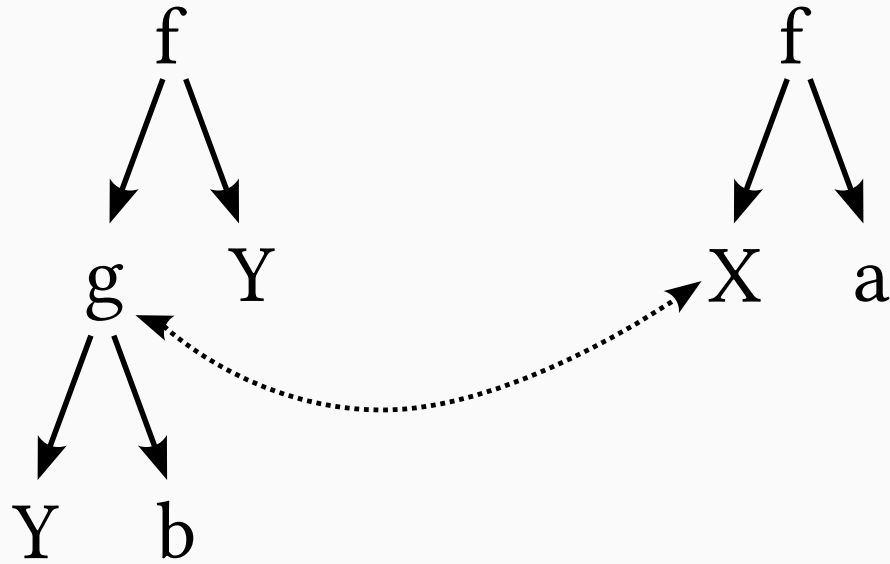
Peut-on unifier le terme $f(g(Y, b), Y)$ avec le terme $f(X, a)$ avec X et Y des variables ?

Unification : Principe

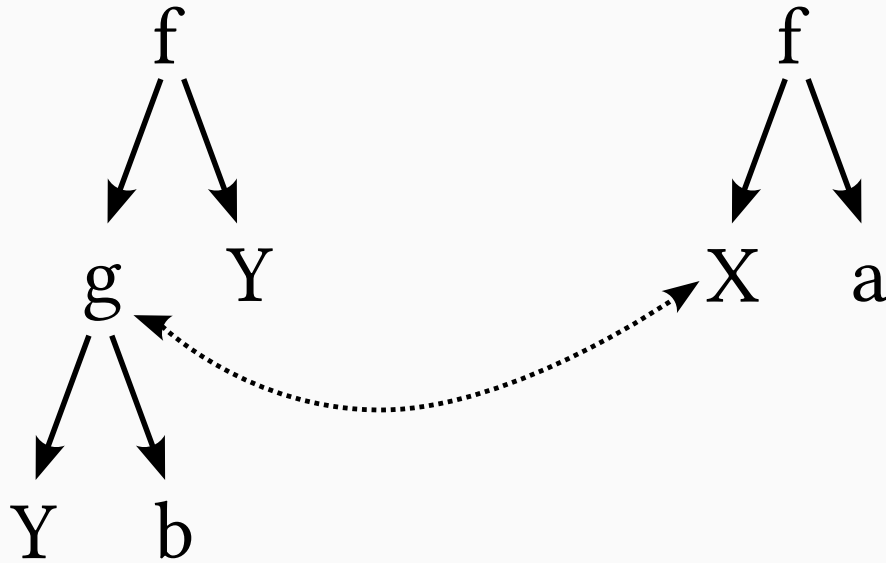


Le foncteur de plus haut niveau est le même.

Unification : Principe

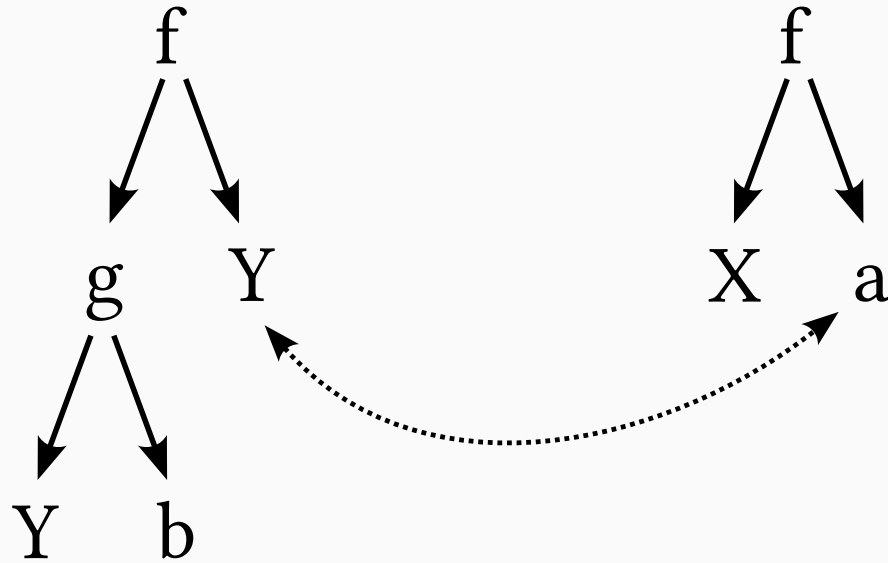


Unification : Principe

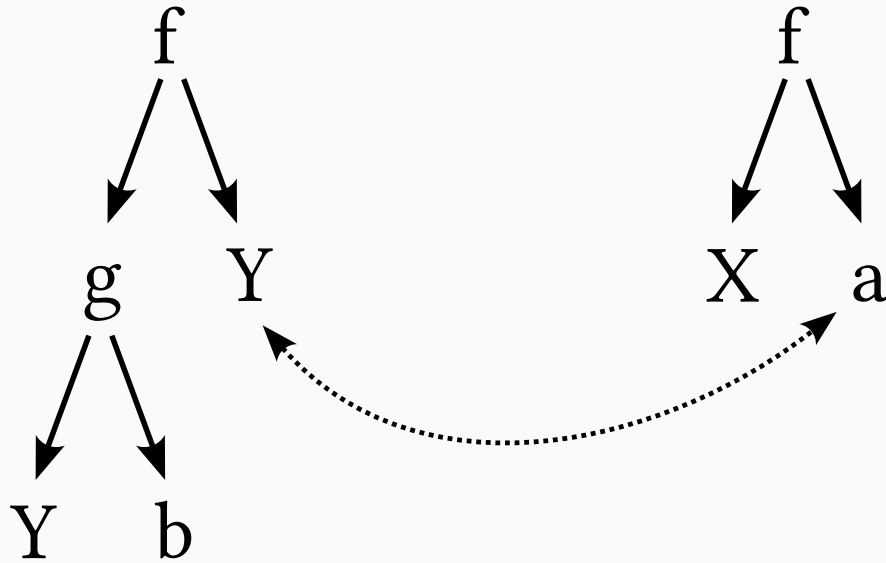


On a une première substitution qui émerge $\sigma = \{X \mapsto g(Y, b)\}$.

Unification : Principe



Unification : Principe



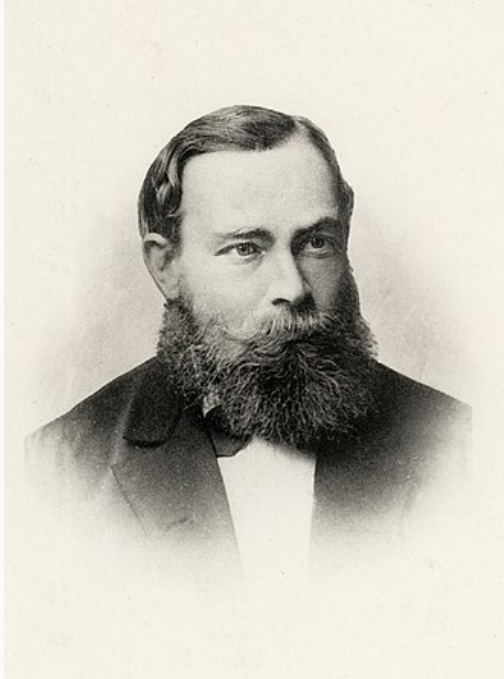
Ce qui complète la substitution $\sigma = \{X \mapsto g(Y, b), Y \mapsto a\}$.

- ▶ L'unification est un procédé incrémental (sans rebroussement)

- ▶ L'unification est un procédé incrémental (sans rebroussement)
- ▶ Traitement des termes récurifs par le **test d'occurrence**.

- ▶ L'unification est un procédé incrémental (sans rebroussement)
- ▶ Traitement des termes récurifs par le **test d'occurrence**.
- ▶ Induit une égalité structurelle naturelle en exhibant σ

Fondations: La logique du premier ordre



La logique du premier ordre à été proposée par Gottlob Frege, un logicien de la fin du XIX^e siècle.



La logique du premier ordre a été proposée par Gottlob Frege, un logicien de la fin du XIX^e siècle.

C'est un système formel dont l'objectif est de décrire des énoncés et de pouvoir raisonner mécaniquement dessus.



La logique du premier ordre à été proposée par Gottlob Frege, un logicien de la fin du XIX^e siècle.

C'est un système formel dont l'objectif est de décrire des énoncés et de pouvoir raisonner mécaniquement dessus.

On parle aussi de **calcul des prédicats du premier ordre**.

► Langage des termes

$X \in \text{Variables}, f \in \text{Symbole}_f$

$t ::= X \mid f \mid f(t_1, \dots, t_n)$

► Langage des termes

$X \in \text{Variables}, f \in \text{Symbole}_f$

$t ::= X \mid f \mid f(t_1, \dots, t_n)$

► Langage des prédicats

$p \in \text{Symbole}_P$

$e ::= \neg e \mid e_1 \wedge e_2 \mid e_1 \vee e_2 \mid e_1 \Rightarrow e_2 \mid p \mid p(t_1, \dots, t_n) \mid \forall X.e \mid \exists X.e$

Logique du premier ordre : définition

► Langage des termes

$X \in \text{Variables}, f \in \text{Symbole}_f$

$t ::= X \mid f \mid f(t_1, \dots, t_n)$

► Langage des prédicats

$p \in \text{Symbole}_P$

$e ::= \neg e \mid e_1 \wedge e_2 \mid e_1 \vee e_2 \mid e_1 \Rightarrow e_2 \mid p \mid p(t_1, \dots, t_n) \mid \forall X.e \mid \exists X.e$

Toute personne a une mère et un père biologique:

$\forall X.(\exists Y.\text{mère}(Y, X)) \wedge (\exists Z.\text{père}(Z, X))$

Système formel avancé pour la manipulation d'expressions logiques:

Système formel avancé pour la manipulation d'expressions logiques:

- Mise en **Forme Prénexe** par regroupement en tête des quantificateurs $\forall$ et $\exists$

Formalisé par David Hilbert et Wilhelm Ackermann en 1928

Système formel avancé pour la manipulation d'expressions logiques:

- Mise en **Forme Prénexe** par regroupement en tête des quantificateurs $\forall$ et $\exists$

Formalisé par David Hilbert et Wilhelm Ackermann en 1928

- Principe de **Herbrandisation** par l'élimination des quantificateurs $\forall$

Système formel avancé pour la manipulation d'expressions logiques:

- ▶ Mise en **Forme Prénexe** par regroupement en tête des quantificateurs $\forall$ et $\exists$

Formalisé par David Hilbert et Wilhelm Ackermann en 1928

- ▶ Principe de **Herbrandisation** par l'élimination des quantificateurs $\forall$ ou
- ▶ Principe de **Skolémisation** par l'élimination des quantificateurs $\exists$

Système formel avancé pour la manipulation d'expressions logiques:

- ▶ Mise en **Forme Prénexe** par regroupement en tête des quantificateurs $\forall$ et $\exists$

Formalisé par David Hilbert et Wilhelm Ackermann en 1928

- ▶ Principe de **Herbrandisation** par l'élimination des quantificateurs $\forall$ ou
- ▶ Principe de **Skolémisation** par l'élimination des quantificateurs $\exists$

Transformations mécanisables

Système formel avancé pour la manipulation d'expressions logiques:

- ▶ Mise en **Forme Prénexe** par regroupement en tête des quantificateurs $\forall$ et $\exists$

Formalisé par David Hilbert et Wilhelm Ackermann en 1928

- ▶ Principe de **Herbrandisation** par l'élimination des quantificateurs $\forall$ ou
- ▶ Principe de **Skolémisation** par l'élimination des quantificateurs $\exists$

Transformations mécanisables mais quid de la Résolution ?

Système formel avancé pour la manipulation d'expressions logiques:

- ▶ Mise en **Forme Prénexe** par regroupement en tête des quantificateurs $\forall$ et $\exists$

Formalisé par David Hilbert et Wilhelm Ackermann en 1928

- ▶ Principe de **Herbrandisation** par l'élimination des quantificateurs $\forall$ ou

- ▶ Principe de **Skolémisation** par l'élimination des quantificateurs $\exists$

Transformations mécanisables mais quid de la Résolution ?

- ▶ Moteur de satisfiabilité modulo théories appelé aussi Solveur SMT ou

Système formel avancé pour la manipulation d'expressions logiques:

- ▶ Mise en **Forme Prénexe** par regroupement en tête des quantificateurs $\forall$ et $\exists$

Formalisé par David Hilbert et Wilhelm Ackermann en 1928

- ▶ Principe de **Herbrandisation** par l'élimination des quantificateurs $\forall$ ou
- ▶ Principe de **Skolémisation** par l'élimination des quantificateurs $\exists$

Transformations mécanisables mais quid de la Résolution ?

- ▶ Moteur de satisfiabilité modulo théories appelé aussi Solveur SMT ou
- ▶ Moteur limité à un sous ensemble : les clauses de Horn !

Expressions constituées de **ou** et de **négation** ayant au plus un littéral positif.

Expressions constituées de **ou** et de **négation** ayant au plus un littéral positif.

Elles ont la forme :

$$\neg a_1 \vee \neg a_2 \vee \dots \vee \neg a_n \vee b$$

Clauses de Horn

Expressions constituées de **ou** et de **négation** ayant au plus un littéral positif.

Elles ont la forme :

$$\neg a_1 \vee \neg a_2 \vee \dots \vee \neg a_n \vee b$$

Qui se transforme (loi de De Morgan) en:

$$\neg(a_1 \wedge a_2 \wedge \dots \wedge a_n) \vee b$$

Clauses de Horn

Expressions constituées de **ou** et de **négation** ayant au plus un littéral positif.

Elles ont la forme :

$$\neg a_1 \vee \neg a_2 \vee \dots \vee \neg a_n \vee b$$

Qui se transforme (loi de De Morgan) en:

$$\neg(a_1 \wedge a_2 \wedge \dots \wedge a_n) \vee b$$

Qui se transforme en:

$$a_1 \wedge a_2 \wedge \dots \wedge a_n \Rightarrow b$$

Programmation en Logique

$\forall x \forall y \forall z \text{ Frère}(y,z) \vee \neg \text{Père}(x,y) \vee \neg \text{Père}(x,z)$ sera représentée

par la clau

+Frère(*x,*

Les variabl

guer des au

disjonction

et l'affirm

Une formule

Dans l'exemple précédent

-Père(*x,*y)

est un littéral

Père(*x,*y)

est une formule atomique



les distin-

signe de

imée par —

un littéral.

Un système de communication homme-machine en Français

Alain Colmerauer, Henri Kanoui, Philippe Roussel & Robert Pasero ~ 1972

Prolog repose sur la resolution de clauses de Horn avec le support de l'unification

Prolog repose sur la resolution de clauses de Horn avec le support de l'unification

L'expression

$$a_1 \wedge a_2 \wedge \dots \wedge a_n \Rightarrow b$$

Prolog repose sur la resolution de clauses de Horn avec le support de l'unification

L'expression

$$a_1 \wedge a_2 \wedge \dots \wedge a_n \Rightarrow b$$

S'écrit avec la syntaxe dite d'Edimbourg:

```
b :- a1, a2, ..., aN.
```

Prolog repose sur la resolution de clauses de Horn avec le support de l'unification

L'expression

$$a_1 \wedge a_2 \wedge \dots \wedge a_n \Rightarrow b$$

S'écrit avec la syntaxe dite d'Edimbourg:

`b :- a1, a2, ..., aN.`

Qui se lit : « b est vrai si a1 et vrai et a2 et vrai etc. jusqu'à aN ».

Assertion

b.

Assertion

b.

Règle

b :- a1, a2, ..., aN.

Assertion

b.

Règle

b :- a1, a2, ..., aN.

But

?- c1, c2, ..., cM.

Assertion

b.

Règle

b :- a1, a2, ..., aN.

But

?- c1, c2, ..., cM.

Comment prouve-t-on un but ?

Assertion

b.

Règle

b :- a1, a2, ..., aN.

But

?- c1, c2, ..., cM.

Comment prouve-t-on un but ? Quid de la résolution ?



John Alan Robinson mathématicien et informaticien anglais, propose un principe de résolution.

Prolog s'appuie sur la **résolution SLD** qui permet la preuve d'une formule à partir d'un ensemble de **clauses de Horn**.

SLD pour Selected, Linear et Definite.

Le processus est cyclique et suit une logique très précise :

Sélection : Choisir le premier but de la question posée.

Le processus est cyclique et suit une logique très précise :

Sélection : Choisir le premier but de la question posée.

Unification : Chercher une règle ou un fait dont la tête s'unifie avec ce but.

Le processus est cyclique et suit une logique très précise :

Sélection : Choisir le premier but de la question posée.

Unification : Chercher une règle ou un fait dont la tête s'unifie avec ce but.

Remplacement : Le but est remplacé par le corps de la règle correspondante.

Le processus est cyclique et suit une logique très précise :

Sélection : Choisir le premier but de la question posée.

Unification : Chercher une règle ou un fait dont la tête s'unifie avec ce but.

Remplacement : Le but est remplacé par le corps de la règle correspondante.

Répétition : Jusqu'à ce qu'il ne reste plus rien ou qu'aucune règle ne corresponde.

Le processus est cyclique et suit une logique très précise :

Sélection : Choisir le premier but de la question posée.

Unification : Chercher une règle ou un fait dont la tête s'unifie avec ce but.

Remplacement : Le but est remplacé par le corps de la règle correspondante.

Répétition : Jusqu'à ce qu'il ne reste plus rien ou qu'aucune règle ne corresponde.

La SLD-résolution utilise deux règles de navigation : l'ordre des littéraux et des clauses.

Lors d'échec, il rebrousse chemin (*backtrack*) jusqu'au dernier choix possible et essaye une autre règle.

Prolog : SLD-résolution par l'exemple

```
p(a).  
p(b).  
q(b).  
r(X) :- p(X), q(X).
```

Prolog : SLD-résolution par l'exemple

p(a).

p(b).

q(b).

r(X) :- p(X), q(X).

?- r(b).

Prolog : SLD-résolution par l'exemple

```
p(a).  
p(b).  
q(b).  
r(X) :- p(X), q(X).
```

```
?- r(b).  
    ↓  
?- p(b), q(b).
```

Prolog : SLD-résolution par l'exemple

```
p(a).  
p(b).  
q(b).  
r(X) :- p(X), q(X).
```

```
?- r(b).  
  ↓  
?- p(b), q(b).  
  ↓  
?- q(b).
```

Prolog : SLD-résolution par l'exemple

p(a).
p(b).
q(b).
r(X) :- p(X), q(X).

?- r(b).
↓
?- p(b), q(b).
↓
?- q(b).
↓
?- $\emptyset$.

Prolog : SLD-résolution avec backtrack par l'exemple

```
p(a).  
p(b).  
q(b).  
r(X) :- p(X), q(X).
```


Prolog : SLD-résolution avec backtrack par l'exemple

p(a).

p(b).

q(b).

r(X) :- p(X), q(X).

?- r(U).

Prolog : SLD-résolution avec backtrack par l'exemple

p(a).
p(b).
q(b).
r(X) :- p(X), q(X).

?- r(U).
↓
?- p(U), q(U).

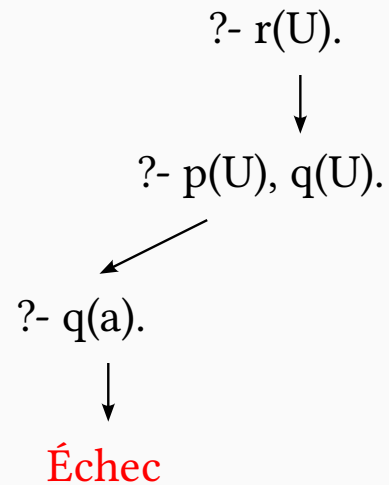
Prolog : SLD-résolution avec backtrack par l'exemple

$p(a).$
 $p(b).$
 $q(b).$
 $r(X) \text{ :- } p(X), q(X).$

$?- r(U).$
↓
 $?- p(U), q(U).$
↙
 $?- q(a).$

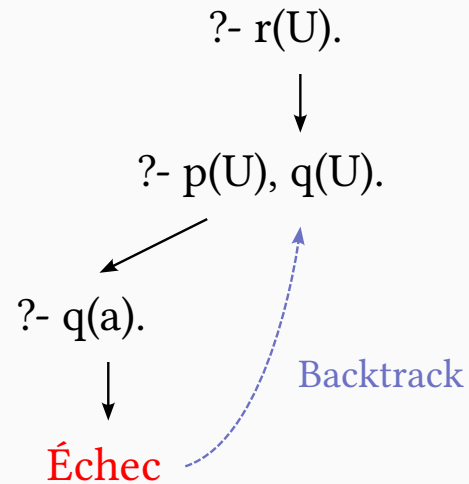
Prolog : SLD-résolution avec backtrack par l'exemple

$p(a).$
 $p(b).$
 $q(b).$
 $r(X) \text{ :- } p(X), q(X).$



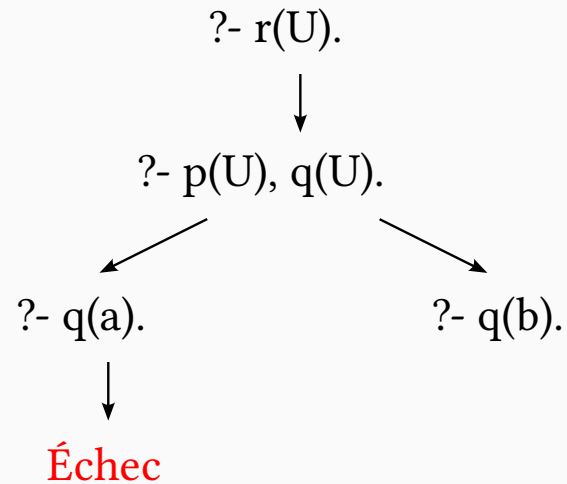
Prolog : SLD-résolution avec backtrack par l'exemple

$p(a).$
 $p(b).$
 $q(b).$
 $r(X) \text{ :- } p(X), q(X).$



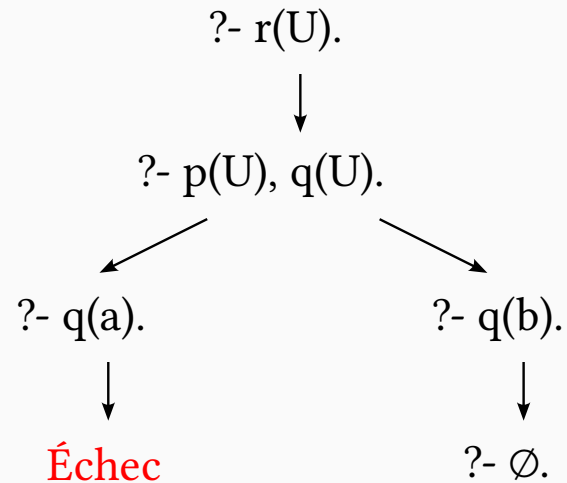
Prolog : SLD-résolution avec backtrack par l'exemple

p(a).
p(b).
q(b).
r(X) :- p(X), q(X).



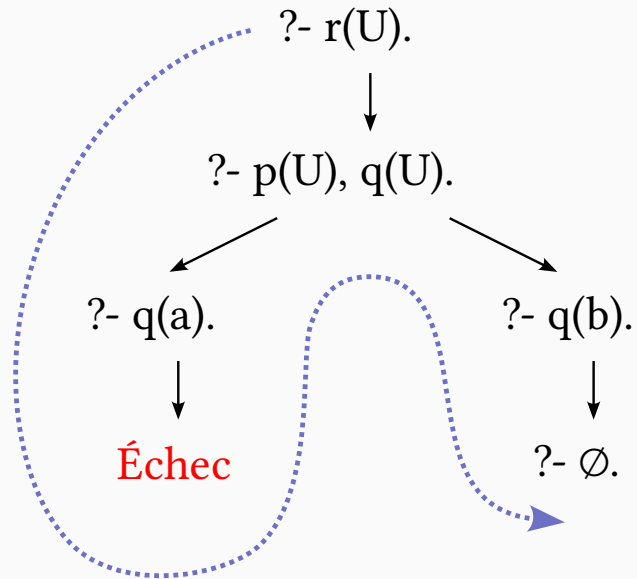
Prolog : SLD-résolution avec backtrack par l'exemple

p(a).
p(b).
q(b).
r(X) :- p(X), q(X).



SLD-résolution avec backtrack

$p(a).$
 $p(b).$
 $q(b).$
 $r(X) \text{ :- } p(X), q(X).$



Prolog : Extensions

Ne pas essayer des règles alternatives pour satisfaire le prédicat courant.

Ne pas essayer des règles alternatives pour satisfaire le prédicat courant.

- ▶ **Assertion native !** appelée *cut* (coupure) en anglais

Ne pas essayer des règles alternatives pour satisfaire le prédicat courant.

- ▶ **Assertion native !** appelée *cut* (coupure) en anglais
- ▶ **Green cut** : sa suppression ne change rien aux réponses

Ne pas essayer des règles alternatives pour satisfaire le prédicat courant.

- ▶ **Assertion native !** appelée *cut* (coupure) en anglais
- ▶ **Green cut** : sa suppression ne change rien aux réponses
- ▶ **Red cut** : sa suppression peut changer les réponses

call(X) lorsque X est unifié à un atome ou un foncteur, il résoud le but en question.

call(X) lorsque X est unifié à un atome ou un foncteur, il résoud le but en question.

💡 Capacité du langage à manipuler ses termes

call(X) lorsque X est unifié à un atome ou un foncteur, il résoud le but en question.

💡 Capacité du langage à manipuler ses termes ... comme Lisp.

call(X) lorsque X est unifié à un atome ou un foncteur, il résoud le but en question.

💡 Capacité du langage à manipuler ses termes ... comme Lisp.

► Principe d'homoiconicité

call(X) lorsque X est unifié à un atome ou un foncteur, il résoud le but en question.

💡 Capacité du langage à manipuler ses termes ... comme Lisp.

► Principe d'homoiconicité

```
and(X,Y) :- call(X), call(Y).  
or(X,Y)  :- call(X), !.  
or(X,Y)  :- call(Y).
```

call(X) lorsque X est unifié à un atome ou un foncteur, il résoud le but en question.

💡 Capacité du langage à manipuler ses termes ... comme Lisp.

► Principe d'homoiconicité

```
and(X,Y) :- call(X), call(Y).  
or(X,Y)  :- call(X), !.  
or(X,Y)  :- call(Y).
```

Cas d'usage d'un **Green Cut**.

call(X) lorsque X est unifié à un atome ou un foncteur, il résoud le but en question.

💡 Capacité du langage à manipuler ses termes ... comme Lisp.

► Principe d'homoiconicité

```
and(X,Y) :- call(X), call(Y).  
or(X,Y)  :- call(X), !.  
or(X,Y)  :- call(Y).
```

Cas d'usage d'un **Green Cut**.

```
?- and(or(true,false), true).
```


Pas d'hypothèse de **monde clos** c-à-d. non monotone.

Pas d'hypothèse de **monde clos** c-à-d. non monotone.

► Combinaison de **call** et de **cut**

Pas d'hypothèse de **monde clos** c-à-d. non monotone.

► Combinaison de **call** et de **cut**

```
not(B) :- call(B), !, fail.  
not(_).
```

Pas d'hypothèse de **monde clos** c-à-d. non monotone.

► Combinaison de **call** et de **cut**

```
not(B) :- call(B), !, fail.  
not(_).
```

Cas d'usage d'un **Red Cut**.

Constraint Logic Programming

Joxan Jaffar[†] and Jean-Louis Lassez

*I.B.M. Thomas J. Watson Research Center
Yorktown Heights
N.Y. 10598
U.S.A*

La programmation par contraintes:

Constraint Logic Programming

Joxan Jaffar[†] and Jean-Louis Lassez

*I.B.M. Thomas J. Watson Research Center
Yorktown Heights
N.Y. 10598
U.S.A*

La programmation par contraintes:

- Extension de la SLD-résolution classique

Constraint Logic Programming

Joxan Jaffar[†] and Jean-Louis Lassez

*I.B.M. Thomas J. Watson Research Center
Yorktown Heights
N.Y. 10598
U.S.A*

La programmation par contraintes:

- Extension de la SLD-résolution classique en
- Intègrant un solveur de contraintes.

Constraint Logic Programming

Joxan Jaffar[†] and Jean-Louis Lassez

*I.B.M. Thomas J. Watson Research Center
Yorktown Heights
N.Y. 10598
U.S.A*

La programmation par contraintes:

- Extension de la SLD-résolution classique en
- Intègrant un solveur de contraintes.

On parle de **CLP modulo théorie** (réels, etc.)

Prolog en action !



Description de la règle d'emprunt capée à 35%

Comment déterminer votre capacité d'emprunt ?

L'établissement bancaire détermine votre **capacité** d'endettement en appliquant à vos **ressources** un taux d'**effort** qui ne doit, en principe, **pas dépasser 35 %**.

Comment déterminer votre capacité d'emprunt ?

L'établissement bancaire détermine votre **capacité** d'endettement en appliquant à vos **ressources** un taux d'**effort** qui ne doit, en principe, **pas dépasser 35 %**.

```
:- use_module(library(clpr)).
```

Comment déterminer votre capacité d'emprunt ?

L'établissement bancaire détermine votre **capacité** d'endettement en appliquant à vos **ressources** un taux d'**effort** qui ne doit, en principe, **pas dépasser 35 %**.

```
:- use_module(library(clpr)).
```

```
detteMaximumPourcent(35).
```

```
dette(famille(Ressources),mensualite(Capacite),dettePourcent(Effort)) :-  
    detteMaximumPourcent(DetteMaxPourcent),  
    { Capacite =< DetteMaxPourcent / 100 * Ressources },  
    { Effort = 100 * Capacite / Ressources }.
```

Vérification d'endettement

```
?- dette(famille(4_000),mensualite(1_000),dettePercentage(25)).
```

Description de la règle d'emprunt capée à 35% ~ Exemples

Vérification d'endettement

```
?- dette(famille(4_000),mensualite(1_000),dettePercentage(25)).
```

```
true
```

Description de la règle d'emprunt capée à 35% ~ Exemples

Vérification d'endettement

```
?- dette(famille(4_000),mensualite(1_000),dettePercentage(25)).
```

true

Calcul de mensualité

```
?- dette(famille(4_000),mensualite(M),dettePercentage(25)).
```

Description de la règle d'emprunt capée à 35% ~ Exemples

Vérification d'endettement

```
?- dette(famille(4_000),mensualite(1_000),dettePercentage(25)).
```

true

Calcul de mensualité

```
?- dette(famille(4_000),mensualite(M),dettePercentage(25)).
```

M = 1_000.0

Description de la règle d'emprunt capée à 35% ~ Exemples

Vérification d'endettement

```
?- dette(famille(4_000),mensualite(1_000),dettePercentage(25)).
```

true

Calcul de mensualité

```
?- dette(famille(4_000),mensualite(M),dettePercentage(25)).
```

M = 1_000.0

Proposition de mensualités à partir d'une dette $\in]20; 25]$

```
?- dette(famille(4_000),mensualite(M),dettePercentage(D)), {20 < D, D =< 25}.
```

Description de la règle d'emprunt capée à 35% ~ Exemples

Vérification d'endettement

```
?- dette(famille(4_000),mensualite(1_000),dettePercentage(25)).
```

true

Calcul de mensualité

```
?- dette(famille(4_000),mensualite(M),dettePercentage(25)).
```

M = 1_000.0

Proposition de mensualités à partir d'une dette $\in]20; 25]$

```
?- dette(famille(4_000),mensualite(M),dettePercentage(D)), {20 < D, D =< 25}.
```

{D=0.025*M, M>800.0, M=<1000.0}

Système de types pour un λ -calcul simple

Système de types pour un λ -calcul simple

La notation $\Gamma \vdash M : T$, où Γ est une liste de paires de la forme $x : B$ où x est une variable et B un type, M est un λ -terme et T un type, se lit « dans le contexte Γ , le terme M a pour type T ».

Système de types pour un λ -calcul simple

La notation $\Gamma \vdash M : T$, où Γ est une liste de paires de la forme $x : B$ où x est une variable et B un type, M est un λ -terme et T un type, se lit « dans le contexte Γ , le terme M a pour type T ».

Représentation d'une règles de typage

Une règle de typage est de la forme:
$$\frac{\Gamma_1 \vdash M_1 : T_1 \dots \Gamma_n \vdash M_n : T_n}{\Gamma \vdash M : T}$$
 et doit se comprendre ainsi: « si $\Gamma_1 \vdash M_1 : T_1 \dots \Gamma_n \vdash M_n : T_n$ alors $\Gamma \vdash M : T$ ».

Système de types pour un λ -calcul simple

La notation $\Gamma \vdash M : T$, où Γ est une liste de paires de la forme $x : B$ où x est une variable et B un type, M est un λ -terme et T un type, se lit « dans le contexte Γ , le terme M a pour type T ».

Représentation d'une règles de typage

Une règle de typage est de la forme:
$$\frac{\Gamma_1 \vdash M_1 : T_1 \dots \Gamma_n \vdash M_n : T_n}{\Gamma \vdash M : T}$$
 et doit se comprendre ainsi: « si $\Gamma_1 \vdash M_1 : T_1 \dots \Gamma_n \vdash M_n : T_n$ alors $\Gamma \vdash M : T$ ».

Règles pour le λ -calcul simplement c-a-d. typé $\lambda \rightarrow$

$$\frac{}{\Gamma_1; x : T; \Gamma_2 \vdash x : T}$$

$$\frac{\Gamma \vdash X : T_1 \rightarrow T_2 \quad \Gamma \vdash Y : T_1}{\Gamma \vdash X Y : T_2}$$

$$\frac{\Gamma, x : T_1 \vdash Y : T_2}{\Gamma \vdash \lambda x. Y : T_1 \rightarrow T_2}$$

Système de types pour un λ -calcul simple

$$\overline{\Gamma_1; x : T; \Gamma_2 \vdash x : T}$$

Système de types pour un λ -calcul simple

$$\overline{\Gamma_1; x : T; \Gamma_2 \vdash x : T}$$

```
system( $\Gamma \vdash X : T$ ) :-  
    atom(X),!,  
    in_gamma(X:T,  $\Gamma$ ).
```

Système de types pour un λ -calcul simple

$$\overline{\Gamma_1; x : T; \Gamma_2 \vdash x : T}$$

$$\frac{\Gamma \vdash X : T_1 \rightarrow T_2 \quad \Gamma \vdash Y : T_1}{\Gamma \vdash X Y : T_2}$$

```
system( $\Gamma \vdash X : T$ ) :-  
    atom(X),!,  
    in_gamma(X:T,  $\Gamma$ ).
```

Système de types pour un λ -calcul simple

$$\frac{}{\Gamma_1; x : T; \Gamma_2 \vdash x : T}$$

$$\frac{\Gamma \vdash X : T_1 \rightarrow T_2 \quad \Gamma \vdash Y : T_1}{\Gamma \vdash X Y : T_2}$$

```
system( $\Gamma \vdash X : T$ ) :-  
    atom(X),!,  
    in_gamma(X:T,  $\Gamma$ ).
```

```
system( $\Gamma \vdash (X @ Y) : T_2$ ) :-  
    system( $\Gamma \vdash X : (T_1 \rightarrow T_2)$ ),  
    system( $\Gamma \vdash Y : T_1$ ).
```

Système de types pour un λ -calcul simple

$$\frac{}{\Gamma_1; x : T; \Gamma_2 \vdash x : T}$$

$$\frac{\Gamma \vdash X : T_1 \rightarrow T_2 \quad \Gamma \vdash Y : T_1}{\Gamma \vdash X Y : T_2}$$

$$\frac{\Gamma, x : T_1 \vdash Y : T_2}{\Gamma \vdash \lambda x. Y : T_1 \rightarrow T_2}$$

```
system( $\Gamma \vdash X : T$ ) :-  
    atom(X),!,  
    in_gamma(X:T,  $\Gamma$ ).
```

```
system( $\Gamma \vdash (X @ Y) : T_2$ ) :-  
    system( $\Gamma \vdash X : (T_1 \rightarrow T_2)$ ),  
    system( $\Gamma \vdash Y : T_1$ ).
```

Système de types pour un λ -calcul simple

$$\frac{}{\Gamma_1; x : T; \Gamma_2 \vdash x : T}$$

$$\frac{\Gamma \vdash X : T_1 \rightarrow T_2 \quad \Gamma \vdash Y : T_1}{\Gamma \vdash X Y : T_2}$$

$$\frac{\Gamma, x : T_1 \vdash Y : T_2}{\Gamma \vdash \lambda x. Y : T_1 \rightarrow T_2}$$

```
system( $\Gamma \vdash X : T$ ) :-  
    atom(X),!,  
    in_gamma(X:T,  $\Gamma$ ).
```

```
system( $\Gamma \vdash (X @ Y) : T_2$ ) :-  
    system( $\Gamma \vdash X : (T_1 \rightarrow T_2)$ ),  
    system( $\Gamma \vdash Y : T_1$ ).
```

```
system( $\Gamma \vdash (X \Rightarrow Y) : (T_1 \rightarrow T_2)$ ) :-  
    atom(X),  
    system(( $\Gamma, X:T_1 \vdash Y : T_2$ )).
```

Fonction identité

```
?- system([]  $\vdash$  (x  $\Rightarrow$  x) : T).
```

Fonction identité

```
?- system([] ⊢ (x ⇒ x) : T).
```

$T = (_A \rightarrow _A)$

Fonction identité

```
?- system([]  $\vdash$  (x  $\Rightarrow$  x) : T).
```

$T = (_A \rightarrow _A)$

Application de fonction sans connaître les hypothèses

```
?- system( $\Gamma \vdash$  (x @ y) : T).
```


Fonction identité

```
?- system([]  $\vdash$  (x  $\Rightarrow$  x) : T).
```

$T = (_A \rightarrow _A)$

Application de fonction sans connaître les hypothèses

```
?- system( $\Gamma \vdash$  (x @ y) : T).
```

$\Gamma = ((_, y:_A), x:(_A \rightarrow T))$

Fonction identité

```
?- system([]  $\vdash$  (x  $\Rightarrow$  x) : T).
```

$T = (_A \rightarrow _A)$

Application de fonction sans connaître les hypothèses

```
?- system( $\Gamma \vdash$  (x @ y) : T).
```

$\Gamma = ((_, y : _A), x : (_A \rightarrow T))$

Fonction d'application (cas de réification)

```
?- system([]  $\vdash$  (x  $\Rightarrow$  y  $\Rightarrow$  (x @ y)) : (T1  $\rightarrow$  T2)).
```

Fonction identité

```
?- system([] ⊢ (x ⇒ x) : T).
```

$T = (_A \rightarrow _A)$

Application de fonction sans connaître les hypothèses

```
?- system(Γ ⊢ (x @ y) : T).
```

$\Gamma = ((_, y:_A), x:(_A \rightarrow T))$

Fonction d'application (cas de réification)

```
?- system([] ⊢ (x ⇒ y ⇒ (x @ y)) : (T1 → T2)).
```

$T1 = T2, T2 = (_A \rightarrow _B)$

Pour terminer



“On dispose de règles du jeu ? on peut alors valider voire inférer des solutions !”

“On dispose de règles du jeu ? on peut alors valider voire inférer des solutions !”

- ▶ Analyse de la Langue Naturelle (NLP)

Analyse syntaxique de phrases, extraction d'entités

“On dispose de règles du jeu ? on peut alors valider voire inférer des solutions !”

- ▶ Analyse de la Langue Naturelle (NLP)

Analyse syntaxique de phrases, extraction d'entités

- ▶ Système expert et Aide à la Décision

Moteur de diagnostic, Système de conformité etc.

“On dispose de règles du jeu ? on peut alors valider voire inférer des solutions !”

- ▶ Analyse de la Langue Naturelle (NLP)

Analyse syntaxique de phrases, extraction d'entités

- ▶ Système expert et Aide à la Décision

Moteur de diagnostic, Système de conformité etc.

- ▶ Problèmes de Satisfaction de Contraintes

Optimisation logistique, Emplois du temps, Chalk

“On dispose de règles du jeu ? on peut alors valider voire inférer des solutions !”

- ▶ Analyse de la Langue Naturelle (NLP)

Analyse syntaxique de phrases, extraction d'entités

- ▶ Système expert et Aide à la Décision

Moteur de diagnostic, Système de conformité etc.

- ▶ Problèmes de Satisfaction de Contraintes

Optimisation logistique, Emplois du temps, Chalk

- ▶ Gestion de Bases de Données Graphes

Réseaux sociaux, Généalogie etc.

Langage polyvalent pour la modélisation de problèmes complexes

Langage polyvalent pour la modélisation de problèmes complexes

- ▶ Programmation déclarative

Langage polyvalent pour la modélisation de problèmes complexes

- ▶ Programmation déclarative
- ▶ Résolution logique (résolution SLD)

Langage polyvalent pour la modélisation de problèmes complexes

- ▶ Programmation déclarative
- ▶ Résolution logique (résolution SLD)
- ▶ Métaprogrammation (homoiconicité)

Langage polyvalent pour la modélisation de problèmes complexes

- ▶ Programmation déclarative
- ▶ Résolution logique (résolution SLD)
- ▶ Métaprogrammation (homoiconicité)
- ▶ Programmation par contraintes (CLP)

Langage polyvalent pour la modélisation de problèmes complexes

- ▶ Programmation déclarative
- ▶ Résolution logique (résolution SLD)
- ▶ Métaprogrammation (homoiconicité)
- ▶ Programmation par contraintes (CLP)
- ▶ Extension au paradigme objet

Langage polyvalent pour la modélisation de problèmes complexes

- ▶ Programmation déclarative
- ▶ Résolution logique (résolution SLD)
- ▶ Métaprogrammation (homoiconicité)
- ▶ Programmation par contraintes (CLP)
- ▶ Extension au paradigme objet
- ▶ Extension au λ -calcul

Quelques réalisations et variantes

- ▶ Sicstus Prolog (Commercial)
- ▶ SWI-Prolog (Open Source) avec un portage en WASM
- ▶ GNU Prolog (Open Source)
- ▶ Embeddable λ Prolog Interpreter (Open Source)
- ▶ Visual Prolog (Commercial)
- ▶ Picat (Freeware)

- ▶ Un système de communication homme-machine en français Alain Colmerauer, Henri Kanoui, Philippe Roussel & Robert Pasero
- ▶ The Art of Prolog - Leon Sterling & Ehud Shapiro
- ▶ Constraint Logic Programming - Joxan Jaffar & Michael J. Maher
- ▶ Warren's Abstract Machine - A TUTORIAL RECONSTRUCTION - Hassan Aït-Kaci
- ▶ Programming with Higher-Order Logic - Dale Miller & Gopalan Nadathur.



<https://github.com/Womate/axanwe>